

TPI OE-TUPaD(3).pdf
PDF

Tengo que hacer un Trabajo Integrador para Organización Empresarial y elegí el proceso de solicitud de vacaciones. ¿Podrías explicarme qué se espera del trabajo y cuál sería una buena forma de abordar el proyecto desde cero?

Sí. Leyendo la consigna, el objetivo principal no es programar algo complejo, sino demostrar que entendés un proceso administrativo, que podés modelarlo con BPMN y que después sos capaz de traducir esa lógica a un chatbot o simulador. TPI OE-TUPaD

Como elegiste **Solicitud de Vacaciones**, en realidad seleccionaste uno de los ejemplos más sencillos y completos para desarrollar.

¿Qué espera la cátedra?

Que actúes como si fueras un consultor tecnológico contratado por una empresa para mejorar un proceso administrativo. TPI OE-TUPaD

Debés entregar:

1. Análisis del proceso actual (AS-IS).
2. Propuesta mejorada (TO-BE).
3. Diagrama BPMN 2.0.
4. Explicación técnica del chatbot.
5. Simulación de una base de datos.
6. Diccionario de datos.
7. Manual de usuario.
8. Casos de prueba y errores.
9. Evidencia del uso de IA.
10. Repositorio GitHub. TPI OE-TUPaD

Cómo arrancar el proyecto desde cero

Paso 1: Definir la empresa

Inventá una empresa sencilla.

Ejemplo:

Empresa: TechSolutions S.A.

Área: Recursos Humanos

Problema: Las solicitudes de vacaciones se realizan por correo electrónico, lo que genera demoras y errores de seguimiento.

Paso 2: Definir el proceso actual (AS-IS)

Describí cómo se hace hoy.

Ejemplo:

1. Empleado envía correo.
2. RRHH revisa saldo.
3. RRHH consulta al supervisor.
4. Supervisor aprueba o rechaza.
5. RRHH informa el resultado.

Problemas:

- Demoras.
- Correos perdidos.
- Falta de seguimiento.
- Errores al calcular días disponibles.

Paso 3: Diseñar el proceso mejorado (TO-BE)

Ahora aparece el chatbot.

Nuevo flujo:

1. Empleado inicia solicitud.
2. Bot pide DNI o legajo.
3. Bot consulta saldo.
4. Bot pide fechas.
5. Bot verifica disponibilidad.
6. Bot envía solicitud al supervisor.
7. Supervisor aprueba o rechaza.
8. Bot informa el resultado.

Paso 4: Crear el BPMN

La consigna pide separar claramente Usuario y Sistema/Bot.

TPI OE-TUPaD

Podrías hacer dos carriles:

Usuario

- Inicia solicitud
- Ingresa fechas
- Recibe respuesta

Sistema/Bot

- Valida empleado
- Consulta saldo
- Verifica días
- Envía solicitud
- Registra resultado

Gateways obligatorios

La consigna pide decisiones lógicas.

TPI OE-TUPaD

Podés usar estas:

Gateway 1

¿Tiene días disponibles?

SI → continúa

NO → rechaza solicitud

Gateway 2

¿Supervisor aprueba?

SI → vacaciones aprobadas

NO → vacaciones rechazadas

Paso 5: Simular la base de datos

La cátedra permite usar una plantilla Excel.

TPI OE-TUPaD

Por ejemplo:

Legajo	Nombre	Días Disponibles
1001	Juan Pérez	15
1002	Ana López	5
1003	Pedro Gómez	20

Y otra hoja:

Solicitud	Legajo	Fecha Inicio	Fecha Fin	Estado
1	1001	01/08	10/08	Aprobada

Paso 6: Máquina de Estados

La consigna le da bastante importancia.

TPI OE-TUPaD

Tenés que demostrar que el bot recuerda en qué paso está cada usuario.

Ejemplo:

Estado 0 → Inicio

Estado 1 → Esperando legajo

Estado 2 → Esperando fecha inicio

Estado 3 → Esperando fecha fin

Estado 4 → Esperando aprobación

Estado 5 → Finalizado

Podés mostrarlo con una tabla.

Paso 7: Camino feliz

El proceso funciona normalmente.

Empleado:

Solicitar vacaciones

Bot:

Ingrese legajo

Empleado:

1001

Bot:

Tiene 15 días disponibles. Ingrese fecha de inicio.

Empleado:

01/08/2026

Bot:

Solicitud enviada al supervisor.

Supervisor:

Aprobar

Bot:

Vacaciones aprobadas.

Paso 8: Camino infeliz

Esto es MUY importante porque lo menciona varias veces la consigna.

TPI OE-TUPaD

TPI OE-TUPaD

Ejemplos:

Legajo inexistente

Usuario:

9999

Bot:

El legajo no existe.

Texto donde debe haber número

Usuario:

Hola

Bot:

Debe ingresar un número de legajo.

Sin saldo de vacaciones

Bot:

Usted posee 0 días disponibles.

Solicitud rechazada.

Fecha inválida

Usuario:

35/15/2026

Bot:

Fecha incorrecta. Utilice DD/MM/AAAA.

Paso 9: Diccionario de Datos

Ejemplo:

Variable	Tipo	Descripción
legajo	int	Identificador del empleado
nombre	string	Nombre del empleado

Variable	Tipo	Descripción
dias_disponibles	int	Días restantes
fecha_inicio	date	Inicio de vacaciones
fecha_fin	date	Fin de vacaciones
estado	string	Estado de solicitud

Paso 10: Manual de Usuario

Algo breve.

Comandos

- Solicitar vacaciones
- Consultar saldo
- Ver estado solicitud

Ejemplo

Usuario:

Solicitar vacaciones

Bot:

Ingrese su legajo

Estructura ideal del PDF (4 a 8 páginas)

1. Introducción.
2. Descripción de la empresa.
3. Problema detectado.
4. Proceso AS-IS.
5. Proceso TO-BE.
6. BPMN.
7. Arquitectura propuesta.
8. Base de datos simulada.
9. Máquina de estados.
10. Casos de prueba.

11. Herramientas de IA utilizadas.

12. Conclusión.

Con el tema de vacaciones, además, podés reutilizar muchísimo material para la defensa porque es un proceso muy fácil de explicar y tiene naturalmente los dos gateways que pide la materia: "¿Tiene saldo?" y "¿El supervisor aprueba?". Eso hace que sea una de las opciones más completas para obtener una buena nota sin complicarte con lógica demasiado avanzada.

     ...  Fuentes

Estoy armando el flujo de un chatbot para gestionar vacaciones. El empleado debe ingresar su legajo, la fecha de inicio y la cantidad de días. ¿Qué validaciones debería tener el sistema y qué decisiones debería contemplar el flujo?

Para que el flujo quede sólido y cumpla con lo que pide la materia (especialmente la parte de **gateways** y **camino infeliz**), no alcanza con pedir legajo, fecha y días. El bot debería validar cada dato y tomar decisiones según las reglas del negocio.

1. Validaciones del legajo

¿El campo está vacío?

Usuario: "" (vacío)

Bot: "Debe ingresar un número de legajo."

¿Es numérico?

Usuario: "Juan"

Bot: "El legajo debe contener únicamente números."

¿Existe en la base de datos?

Usuario: "9999"

Bot: "No se encontró un empleado con ese legajo."

Gateway:

¿Legajo válido?

- Sí → Continúa
- No → Solicita nuevamente el legajo

2. Validaciones de la fecha de inicio

¿Tiene formato correcto?

Usuario: 35/15/2026

Bot: "La fecha ingresada no es válida."

¿La fecha ya pasó?

Usuario: 01/01/2026

(Suponiendo que hoy es junio)

Bot: "La fecha de inicio debe ser posterior a la fecha actual."

¿Se encuentra dentro de un período permitido?

Por ejemplo, la empresa exige solicitar vacaciones con al menos 15 días de anticipación.

Gateway:

¿Cumple la anticipación mínima?

- Sí → Continúa
- No → Rechaza solicitud

3. Validaciones de cantidad de días

¿Es un número?

Usuario: "muchos"

Bot: "Debe ingresar una cantidad numérica de días."

¿Es mayor que cero?

Usuario: 0

Bot: "La cantidad de días debe ser mayor a cero."

¿No supera un límite razonable?

Usuario: 500

Bot: "Cantidad de días inválida."

4. Validaciones de negocio (las más importantes)

¿Tiene saldo suficiente?

Supongamos:

Empleado → 10 días disponibles

Solicita → 15 días

Gateway:

¿Días solicitados $\leq$ días disponibles?

- Sí → Continúa
- No → Rechaza solicitud

Esta es una de las decisiones más importantes para el BPMN.

¿Existe superposición con otras vacaciones?

Ejemplo:

Ya tiene vacaciones del 10 al 20 de julio.

Solicita del 15 al 25 de julio.

Gateway:

¿Existe una solicitud que se superpone?

- Sí → Rechaza
- No → Continúa

¿El supervisor aprueba?

Después de registrar la solicitud.

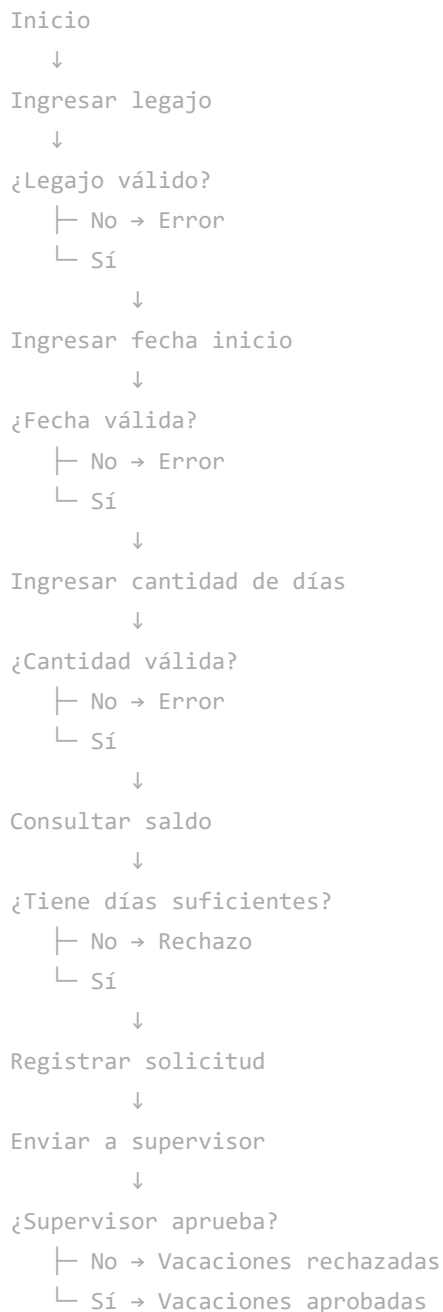
Gateway:

¿Supervisor aprueba?

- Sí → Vacaciones aprobadas
- No → Vacaciones rechazadas

Esta es la segunda compuerta fuerte que te pide la consigna.

Flujo completo sugerido





Fin

Si querés destacarte en la defensa

Agregaría una tercera decisión:

¿La cantidad solicitada supera 10 días?

- No → Aprobación automática.
- Sí → Requiere aprobación del supervisor.

Eso hace que el BPMN tenga más lógica de negocio y demuestra mejor el uso de gateways sin complicar demasiado el proyecto.



Necesito desarrollar el programa en Python. ¿Cómo me recomendás organizar el código para que quede ordenado y fácil de mantener? ¿Qué funciones debería crear?

Teniendo en cuenta los trabajos que venís haciendo en la TUP, te recomiendo **no complicarte con clases ni programación avanzada**. La mejor opción es un programa modular con funciones simples. Va a quedar ordenado, fácil de explicar en la defensa y acorde al nivel de la materia.

Estructura general

```
# Variables / Base de datos simulada
```

```
# Funciones de validación
```

```
# Funciones del proceso
```

```
# Menú principal
```

```
""" Ejecutar programa """
```

```
Python
```

```
Ejecutar
```

1. Base de datos simulada

Podés usar diccionarios.

```
empleados = {  
    1001: {"nombre": "Juan", "dias": 15},  
    1002: {"nombre": "Ana", "dias": 5},  
    1003: {"nombre": "Pedro", "dias": 20}  
}
```

Python

Ejecutar

Esto además te sirve para demostrar la "persistencia de datos simulada" que menciona la consigna.

2. Funciones de validación

Estas funciones se encargan únicamente de verificar datos.

Validar legajo

```
def validar_legajo(legajo):
```

Python

Ejecutar

Validar fecha

```
def validar_fecha(fecha):
```

Python

Ejecutar

Validar cantidad de días

```
def validar_dias(dias):
```

Python

Ejecutar

3. Funciones de consulta

Consultar saldo

```
def consultar_saldo(legajo):
```

Python

Ejecutar

Obtener nombre

```
def obtener_nombre(legajo):
```

Python

Ejecutar

4. Funciones del proceso principal

Acá está la lógica del chatbot.

Solicitar vacaciones

```
def solicitar_vacaciones():
```

Python

Ejecutar

Dentro de esta función:

1. Pedís legajo.
2. Validás.
3. Pedís fecha.
4. Validás.
5. Pedís días.
6. Validás.
7. Consultás saldo.
8. Registrás solicitud.
9. Mostrás resultado.

Aprobar solicitud

```
def aprobar_solicitud():
```

Python

Ejecutar

Puede simular la decisión del supervisor.

Mostrar solicitudes

```
def mostrar_solicitudes():
```

Python

Ejecutar

Para ver las solicitudes registradas.

5. Funciones de interfaz

Por ejemplo:

```
def mostrar_menu():  
    print("1. Solicitar vacaciones")  
    print("2. Consultar saldo")  
    print("3. Ver solicitudes")
```

Python

Ejecutar

6. Programa principal

```
def main():  
  
    while True:  
  
        mostrar_menu()  
  
        opcion = input("Seleccione una opción: ")  
  
        if opcion == "1":  
            solicitar_vacaciones()
```

```
elif opcion == "2":
    consultar_saldo_usuario()

elif opcion == "3":
    mostrar_solicitudes()

elif opcion == "4":
    print("Fin del programa")
    break

else:
    print("Opción inválida")
```

Organización ideal para tu TP

vacaciones.py

```
├─ empleados (diccionario)
├─ solicitudes (lista)

├─ validar_legajo()
├─ validar_fecha()
├─ validar_dias()

├─ consultar_saldo()
├─ obtener_nombre()

├─ solicitar_vacaciones()
├─ aprobar_solicitud()
├─ mostrar_solicitudes()

├─ mostrar_menu()

└─ main()
```

Algo que te recomiendo agregar

Como la consigna habla de **máquina de estados**, podrías reflejarla dentro de `solicitar_vacaciones()` con comentarios:

```
# Estado 1: Solicitar legajo

# Estado 2: Solicitar fecha

# Estado 3: Solicitar cantidad de días
```



```
# Estado 4: Verificar saldo
```

```
# Estado 5: Aprobar/Rechazar
```

Python

Ejecutar

Aunque el programa sea simple, después en la documentación vas a poder decir que el chatbot avanza por distintos estados del proceso, que es exactamente lo que pide la materia. Esto te suma bastante en la parte teórica sin tener que programar algo mucho más complejo.

 Fuentes

Ya terminé el programa y ahora necesito redactar el README del proyecto. ¿Qué secciones debería incluir para que la documentación quede completa y profesional?

Para este trabajo en particular, el README no debería ser solamente una explicación técnica del código. Como la materia es **Organización Empresarial**, conviene que combine la parte funcional del chatbot con la parte de negocio y BPMN.

Yo lo estructuraría así:

Título del Proyecto

```
# Sistema de Gestión de Solicitud de Vacaciones
```

```
Trabajo Práctico Integrador
```

```
Materia: Organización Empresarial
```

Markdown

Descripción General

Explicá brevemente qué hace el proyecto.

Ejemplo:

Descripción

Este proyecto simula el proceso de gestión de solicitudes de vacaciones mediante un chatbot c

El sistema permite a los empleados ingresar su legajo, solicitar días de vacaciones y recibir

La solución fue diseñada a partir de un modelo BPMN 2.0, buscando automatizar un proceso admi

Markdown

Objetivo

Objetivo

Automatizar el proceso de solicitud de vacaciones, reduciendo errores administrativos y mejor

Markdown

Proceso Modelado

Acá podés resumir el flujo de negocio.

Flujo del Proceso

1. El empleado inicia una solicitud.
2. Ingresa su legajo.
3. El sistema verifica la existencia del empleado.
4. El sistema consulta los días disponibles.
5. El empleado ingresa la fecha de inicio.
6. El empleado indica la cantidad de días solicitados.
7. El sistema valida la solicitud.
8. La solicitud es aprobada o rechazada.
9. Se informa el resultado al empleado.

Tecnologías Utilizadas

Tecnologías Utilizadas

- Python 3
- Diccionarios
- Listas
- Funciones
- Estructuras condicionales
-

Markdown

Si usaste GitHub:

- Git+

Markdown

Estructura del Programa

Esta parte suele gustar mucho a los profesores.

Estructura del Programa

El sistema se encuentra dividido en funciones para facilitar su mantenimiento:

- validar_legajo()
- validar_fecha()
- validar_dias()
- consultar_saldo()
- solicitar_vacaciones()
- mostrar_solicitudes()
- mostrar_menu()

Base de Datos Simulada

La consigna menciona persistencia de datos simulada.

Base de Datos Simulada

La información de los empleados se almacena mediante diccionarios que contienen:

- Número de legajo
- Nombre del empleado
- Días de vacaciones disponibles

Markdown

Validaciones Implementadas

Muy importante para la defensa.

Validaciones

El sistema contempla las siguientes validaciones:

- Legajo existente.
- Campos obligatorios.
- Cantidad de días válida.
- Disponibilidad suficiente de días.

Markdown

Ejemplo de Uso

Ejemplo de Uso

Empleado: Solicitar vacaciones

Sistema: Ingrese su legajo

Empleado: 1001

Sistema: Tiene 15 días disponibles

Empleado: Fecha de inicio: 01/08/2026

Empleado: Cantidad de días: 5

Sistema: Solicitud aprobada

Instalación y Ejecución

Instalación

1. Descargar el proyecto.
2. Abrir una terminal.
3. Ejecutar:

Markdown

BPMN

Como el BPMN es parte fundamental del trabajo, agregaría una sección exclusiva.

Diagrama BPMN

El flujo del sistema fue modelado mediante BPMN 2.0.

El diagrama contempla:

- Eventos de inicio y fin.
- Carriles de Usuario y Sistema.
- Compuertas de decisión.
- Caminos alternativos de aprobación y rechazo.
- ...

Podés incluso insertar una imagen del BPMN en el README.

Herramientas de IA Utilizadas

La consigna lo pide explícitamente.

Markdown

Herramientas de IA Utilizadas

Durante el desarrollo se utilizaron herramientas de inteligencia artificial para:

- Diseño del algoritmo.
- Revisión de validaciones.
- Optimización del flujo del chatbot.
- Elaboración de documentación.

Herramientas utilizadas:

- ChatGPT
- DeepSeek

Autor

Markdown

Autor

Brisa Chirino

Trabajo Práctico Integrador

Organización Empresarial

Tecnicatura Universitaria en Programación a Distancia

Con esta estructura te queda un README bastante profesional, similar al que se encontraría en un proyecto real de GitHub, pero además alineado con todos los puntos que la cátedra evalúa.



¿Te gusta este tipo de personalidad?

